



Your AI should be a staff, not a search box.

Many purpose-built agents · on hardware you own · thinking with the AI you choose · reachable by the people you let in · through Telegram.

$$\text{OpenClaw} + \text{Docker} + \text{Telegram} = \text{Hatchabot}$$

the open-source agent runtime · one isolated container per agent · the front door everyone already has

THE PROBLEM

A brilliant model, in the wrong shape

You already pay for a model that can do almost anything. What you get is **one assistant, in one app, from one company, for one person** — whose memory of you is whatever that company decides to keep.

- **"I keep re-explaining myself."** Every conversation starts from zero.
- **"One assistant can't be good at everything."** Your tax questions, your kids' homework and your condo board all get the same generic voice.
- **"I don't want to be locked in."** Your history, your prompts, your habits live in someone else's product.
- **"Only I can use it."** Your family can't message it; it doesn't do anything when you're not looking.

Chat apps are sessions you visit

Open the app, type, close it, forget. The model is powerful; the container around it is a search box with a scroll bar.

Hatchabot agents are things you own

Files on your disk with a job, a personality and a memory that grows for years — behind a contact your whole household can message.

THE IDEA

The opposite shape

Many agents, each with a job

A kitchen helper, a tax advisor, a homework tutor, a scheduler that reads its own inbox. Made in a minute: a name, a paragraph, a bot token.

On a computer you own

Every agent is a container on your machine. Memory, credentials and history stay home. Pair it with a local model and nothing leaves at all.

With the AI you choose

Claude (subscription or API), Gemini, or a local model — per agent, switchable live. Cheap models for simple jobs, the best for the ones that matter.

For the people you let in

Invite family by link or QR. Several people can talk to one agent — privately, or together in a group room — with memory shared or personal, and everyone told which.

Through Telegram

The app everyone already has. Each agent is a real contact with a name and a face. Nothing to install, no accounts to create.

Run as a fleet

Not a chatbot — a control plane: health, usage, audit trail, backups, security posture, upgrades rolled out one agent at a time.

OpenClaw + Docker = Hatchabot. Every agent is a stock **OpenClaw** gateway — the open-source agent runtime with tools, memory search and channels — running in its own **Docker** container with its own volume, bot and credentials. Hatchabot is OpenClaw *new and improved for a household*: many isolated installs, run as a fleet, by more than one person, with the lifecycle and safety OpenClaw deliberately leaves to you.

What we believe an agent is

Agents are things you own

Each one is a container with a persona (`SOUL.md`), operating rules (`AGENTS.md`) and memory (`MEMORY.md`) — plain files you can read and edit, snapshotted before every change.

An agent persists across restarts, rebuilds, model switches and moves between machines. What it knows about you accumulates for years, and it's yours.

Memory is a file, not a chat log

The agent writes durable facts as it goes. Save a conversation to memory. Download its whole chat history. Recover the context of a thread that got reset.

Nothing important is trapped in a transcript.

Not locked to one lab

Anthropic, Google, or a local server with no account at all — per agent, switchable live without a rebuild.

Group agents into **classes** and retune a whole tier in one place. Your agents outlive any provider's pricing decision.

How it should behave

The front door is Telegram

No app to ship, no accounts to run, no notification system to build. Bots are first-class in Telegram: deep links, group chats, per-agent allowlists — for free.

Isolation by construction

One container, one bot, one set of credentials per agent. Give an agent exactly the data it needs — a folder, a git repo, a Google account with "read mail but never send" — and nothing else.

A daily posture check names every agent that has both a wide audience and a powerful capability.

Reversible, or honest about not being

Snapshots before edits and rebuilds. Rollback on a failed move. A full audit timeline. Usage reported as tokens, not a made-up bill. Health probes that distinguish "listed as running" from "actually answering".

Claude Max and Telegram, on purpose

Claude Max: consumer pricing for a fleet

A Max subscription is a flat monthly plan. `claude setup-token` turns it into a portable credential Hatchabot injects into every agent — one token per account, revocable, never a shared login.

So a household of always-on agents — schedulers polling inboxes, advisors reading mail, a dozen personas — runs on **one flat fee** instead of per-token metering. For this kind of fleet that is the difference between *affordable* and *a bill you watch*.

Telegram: 20 real identities per account

Free, on every platform, and bots are a first-class feature. BotFather makes one in a minute; a free account can own 20, Premium 40, and every household member adds more.

Each agent gets a separate identity people already know how to message. The alternative — shipping our own app with our own accounts and push notifications — would cost more than the whole rest of the product.

FEATURES

What you can do that you can't do with a chat app

CAPABILITY	IN PRACTICE
Create agents fast	Name + one paragraph + a bot token → a running, remembering agent in a minute. Clone one you like; import a shared template; keep a launchpad of planned agents.
Many humans, one agent	Private one-to-one threads with the same agent, or a shared room. Invite by link or QR. Shared or private memory, declared.
Choose the brain per agent	Claude, Gemini or a local model; switch the model live; classes for tiers; migrate a whole source at once.
Give agents accounts and data	Gmail / Calendar / Drive / Sheets per agent (with "no send"); read-only or writable folders; git repos it commits to; per-agent secrets.
Schedule and react	Cron tasks, event-triggered tasks (a zero-cost check decides whether to wake the model), run-now tests.
Agents that consult each other	Grant an investing agent access to your tax and legal agents; it asks them mid-task, with scoped tokens, loop guards and rate limits.
Master → child lineage	A master agent's improvements distil into proposals that push to its children.
Move, copy, share, back up	Rehost to another machine in one step; download as one file; share as a template with no secrets; nightly backups with a restore drill.
Rebuild without fear	Rebuilds keep memory. Only a change of AI source resets a thread — and that offers to save memory first, and to recover context after.

Operator-grade, because agents that matter need it

See what's going on

- Fleet health: probed, not assumed
- Usage per agent per day, tokens/hour
- Audit timeline of every change
- Daily security posture with diffs
- Legend with every pending action

Change things safely

- Bulk actions by source, group, class, state
- Classes: model + source + image, retuned in one place
- Runtime images: build a candidate, try it on **one** agent, promote to the fleet, or discard
- Snapshots before every edit; rollback on failed moves

Keep it yours

- Nightly backups: every volume, the registry, the key
- Restore one agent from any set
- Export / import as a single file
- Runners: place agents on a second machine
- Everything works over a private Tailscale network — nothing opened to the internet

COMPARED

Versus Claude.ai, ChatGPT and Codex

	CLAUDE.AI / CHATGPT	CODEX (AND CODING AGENTS)	HATCHABOT
Shape	One assistant, one app, one person	One agent, one repo, one task at a time	A fleet of agents with jobs, for a household
Runs where	Their cloud	Their cloud or your terminal	Your machine (containers), reachable over your private network
Memory	What the provider keeps; opaque	The repo and the session	Files you read, edit, back up; survives rebuilds and moves
Who can talk to it	You, in the app	You, in the terminal or PR	Anyone you invite, on Telegram; private threads or group rooms
Choice of AI	That company's models	That company's models	Claude, Gemini, local — per agent, switchable live
Acts on its own	Limited scheduled tasks	When you run it	Cron and event-triggered tasks; polls its own inbox; consults other agents
Access to your things	Connectors chosen by the vendor	The repo	Per agent: a folder, a git repo, a Google account with limits you set
Cost model	Per seat	Per token / per seat	One Claude Max plan powers every agent; local models cost nothing per token

Not a replacement for the models — Hatchabot *uses* Claude. It replaces the app around it. A coding agent with a git repo is one of the things you can make.

COMPARED

OpenClaw, new and improved: what Hatchabot adds

Every Hatchabot agent **is** an OpenClaw gateway. Hatchabot adds the layer OpenClaw deliberately doesn't have: many installs, run as a fleet, by more than one person.

One install per agent, so agents can't hurt each other

Plain OpenClaw is one process, one config, one workspace and credential set shared by every agent in it. Hatchabot gives each agent its own container, volume, bot token, tool policy and secrets. A runaway task, a bad edit, a rate-limited source or a crash touches one agent — the other thirty keep answering.

Upgrades the same way

The runtime image is pinned and rolled out per agent, candidate first — never "upgrade the gateway and hope every agent survives".

Lifecycle instead of hand-editing

Create, clone, rebuild, archive, restore, delete, move — with a snapshot before every change. In plain OpenClaw that's you, a shell and a directory.

More than one human

Owners, members, invites, pairing approvals, group rooms, shared-vs-private memory people are told about. OpenClaw's allowlist is a config line.

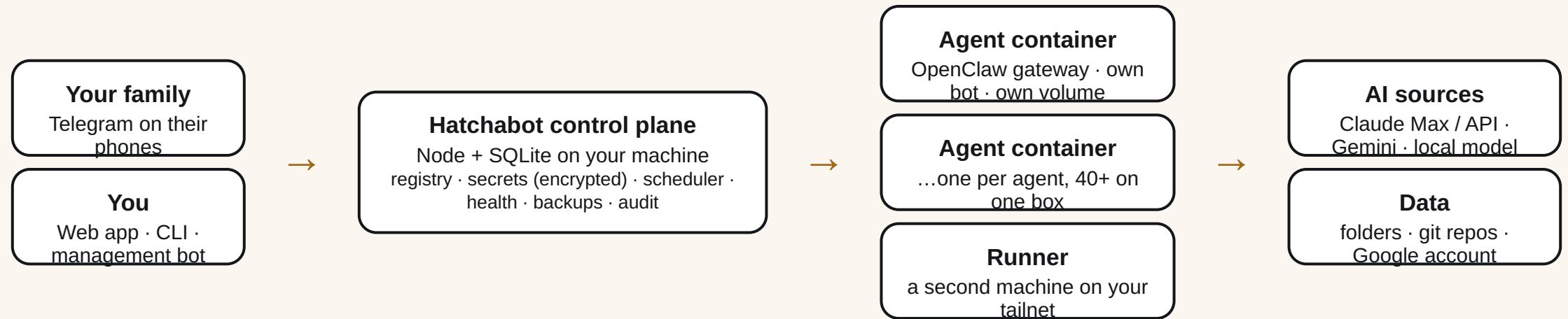
Credentials managed once

One setup-token serves every agent; Google accounts attach per agent; secrets are write-only.

If you run one agent for yourself, plain OpenClaw is fine. Hatchabot is for when there are several, they matter, and other people talk to them.

HOW IT WORKS

One control plane, one container per agent



Persisted

Each agent's `$HOME` is a Docker volume: persona, memory, sessions, tools. The runtime image is shared, versioned, and swapped under an agent without touching the volume.

Private by default

The web app binds to your machine; reach it from anywhere over Tailscale, never a port on your router. Telegram works from anywhere regardless.

Open source

TypeScript, ~950 tests, MIT. Fourteen full audits so far. Install from source in one command; upgrade by tag with automatic rollback.

GET STARTED

Your first agent in 15 minutes

- 1 Make a bot.** Telegram → @BotFather → `/newbot`.
Keep the token. (2 min)
- 2 Get Claude ready.** `npm i -g @anthropic-ai/claude-code`, log in once, `claude setup-token`. (3 min)
- 3 Install.** `git clone ... hatchabot && cd hatchabot && ./scripts/setup-host.sh` — checks Docker and Node, builds the runtime, installs the service, links the CLI. (5 min, mostly waiting)
- 4 Connect the AI.** Open the app, paste the setup token on the Welcome card. (1 min)
- 5 Create the agent.** Tap +, name it, one paragraph about its job, paste the bot token. (1 min)
- 6 Say hi.** Open the Telegram link and send a message. It will still remember this conversation next year.

You need

- A computer that stays on — Linux or macOS, Docker, Node 22+ (a Mac mini, a home server, an old laptop)
- A Telegram account
- A Claude subscription (Pro works for a couple of agents; Max for a household) — or an API key, or a local model with no account at all

Then, optionally

- Tailscale, to open the app from anywhere without exposing anything
- Google connections, folders, git repos, schedules, peers



For people who want more than one assistant — shared with a household, on their own machine, with their own choice of AI.

github.com/hatchabot/hatchabot · hello@hatchabot.com · MIT licensed